



SISTEMA DE VIGILANCIA

Por:

- CARLOS NICOLAS MORALES CUELLAR (23190329)
- JOSE MARTIN RAMOS CALZADA (231903417)
- ROJAS CRISOSTOMO RONALD ENRIQUE(23190346)

RESUMEN DEL PROYECTO

Enfoque Colaborativo y de Bajo Costo: Este proyecto presenta el diseño e implementación de un sistema de vigilancia IoT de código abierto (open-source). Su propósito es ofrecer una plataforma base, accesible y bien documentada, para que estudiantes y desarrolladores puedan clonar, adaptar y expandir el sistema según sus propias necesidades.



Eficiencia mediante Edge Computing: A diferencia de sistemas tradicionales, el procesamiento de las activaciones de movimiento se realiza localmente en nodos de bajo costo (Raspberry Pi Zero 2W y ESP32-CAM) equipados con sensores PIR.



Gestión de Alertas y Ancho de Banda: Un servidor local desarrollado en Flask actúa como intermediario, gestionando la comunicación LAN y enviando alertas asíncronas hacia la API de Telegram. Esto evita enviar video constante a la nube, optimizando el consumo de red.



Introducción – Contexto y Problemática

La Inseguridad y el Costo Tecnológico: El aumento constante de la inseguridad ciudadana en el Perú exige soluciones de monitoreo accesibles. Sin embargo, la dependencia exclusiva de sistemas comerciales limita la implementación en zonas o proyectos con recursos restringidos.



Saturación del Canal de Subida (Uplink): Las soluciones convencionales requieren la transmisión continua de flujos de video hacia servidores en la nube (consumiendo entre 2 y 4 Mbps fijos por cámara). Este modelo centralizado satura el ancho de banda de subida de las redes residenciales, afectando el rendimiento de otros usuarios.

La Alternativa: Desarrollar una arquitectura descentralizada y colaborativa que procese las detecciones localmente. Al enviar alertas fotográficas en ráfagas cortas únicamente ante eventos reales, se libera el espectro inalámbrico y se evita la dependencia de ecosistemas cerrados.



Objetivos

Objetivo General:

- Diseñar e implementar una plataforma IoT de vigilancia colaborativa basada en Edge Computing, que optimice el uso del espectro inalámbrico y garantice la transparencia total del tráfico.

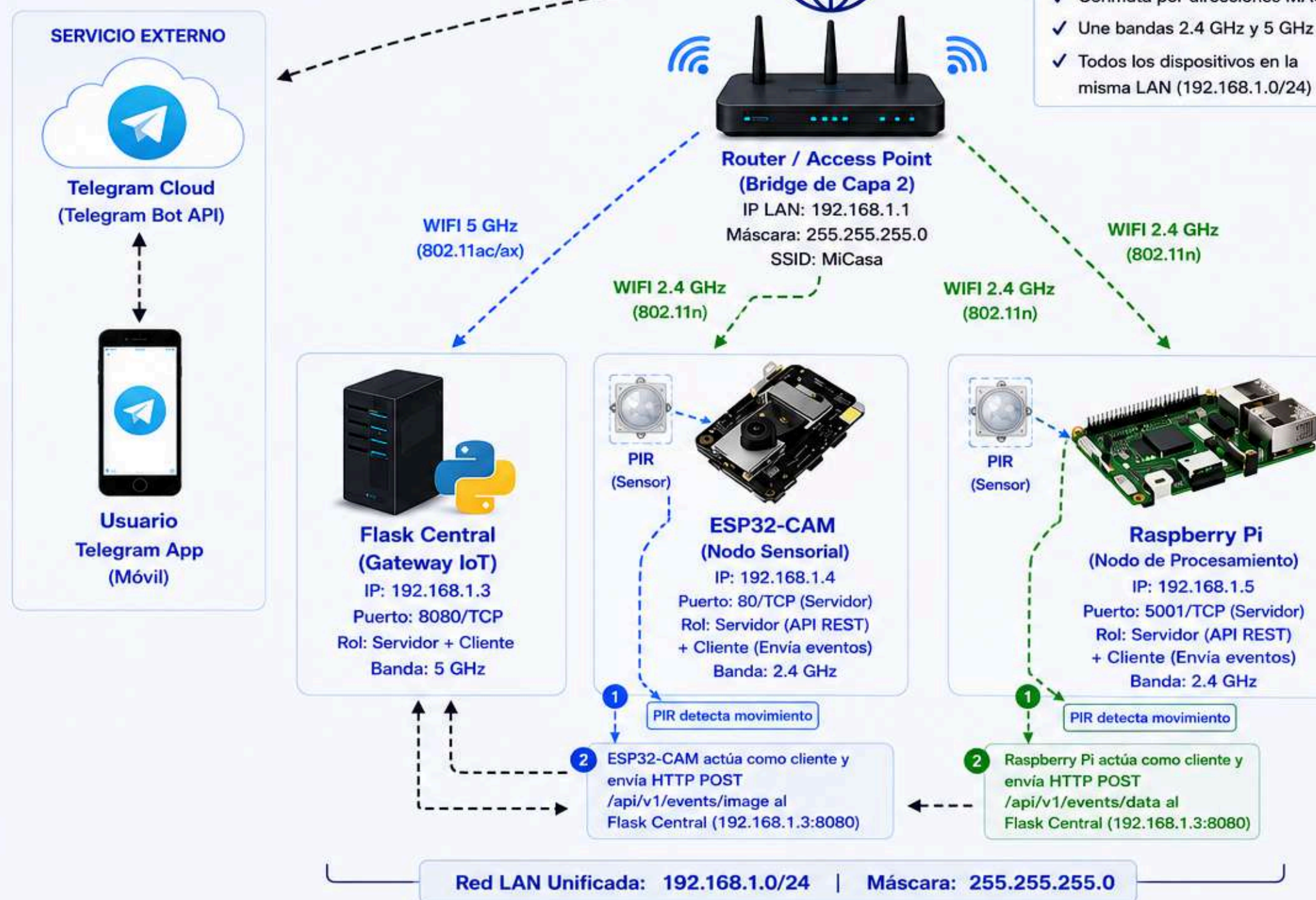
Objetivos Específicos:

- **Despliegue en el Borde:** Implementar una topología modular utilizando microcontroladores y microprocesadores para descentralizar la captura multimedia y asegurar la escalabilidad del hardware.
- **Desacoplamiento Lógico:** Desarrollar un servidor middleware local (Flask) que aíse la capa de percepción y gestione de forma segura las peticiones hacia APIs externas.
- **Auditoría de Protocolos:** Validar el rendimiento del sistema analizando métricas reales (latencia, throughput) y tráfico mediante inspección profunda de paquetes (DPI) ante eventos de activación física.

Marco Teórico - Topología

1 TOPOLOGÍA DE RED

El Access Point (AP) actúa como un bridge de capa 2 (IEEE 802.11) que interconecta las estaciones inalámbricas (2.4 GHz y 5 GHz) dentro de la misma LAN 192.168.1.0/24.



NOTAS IMPORTANTES

- Todos los dispositivos (Flask, ESP32-CAM, Raspberry Pi) están en la misma subred y pueden comunicarse directamente.
- El AP no enruta (no opera en Capa 3), solo conmuta tramas en Capa 2.

- Los servicios (APIs) usan HTTP sobre TCP para intercambio de datos.
- Puertos de servicio del sistema:
 - Flask Central: 8080/TCP (HTTP API)
 - ESP32-CAM: 80/TCP (HTTP API)
 - Raspberry Pi: 5001/TCP (HTTP API)

- Los clientes (Flask como cliente o nodos como clientes) usan puertos efímeros asignados por el sistema operativo.
- La topología permite una arquitectura orientada a eventos y comunicación cliente-servidor sobre TCP/IP.

LEYENDA DE ENLACES

- Enlace WiFi 5 GHz (802.11ac/ax)
- Enlace WiFi 2.4 GHz (802.11n)
- Enlace Ethernet (Alámbrico)
- Comunicación TCP/IP (L3/L4)
- Bridge de Capa 2 (802.11)

PARÁMETROS DE LA LAN

Red	192.168.1.0/24
Máscara	255.255.255.0
Gateway (AP)	192.168.1.1
DNS (Router)	192.168.1.1
DHCP	Habilitado en el Router/AP
Rango DHCP	192.168.1.100 - 192.168.1.200

CONCEPTOS CLAVE APLICADOS

- Bridge de Capa 2 (802.11)**
El AP conmuta tramas basándose en direcciones MAC, interconectando todas las estaciones dentro de la misma LAN.
- WLAN (IEEE 802.11)**
Conexiones inalámbricas en bandas de 2.4 GHz (802.11n) y 5 GHz (802.11ac/ax).
- LAN (Red de Área Local)**
Todos los dispositivos pertenecen a la misma red 192.168.1.0/24.
- ARP (Resolución de Direcciones)**
Los dispositivos usan ARP para resolver IP → MAC dentro de la LAN.
- TCP/IP y HTTP**
La comunicación entre nodos y servicios externos usa TCP como transporte y HTTP como protocolo de aplicación (REST API).

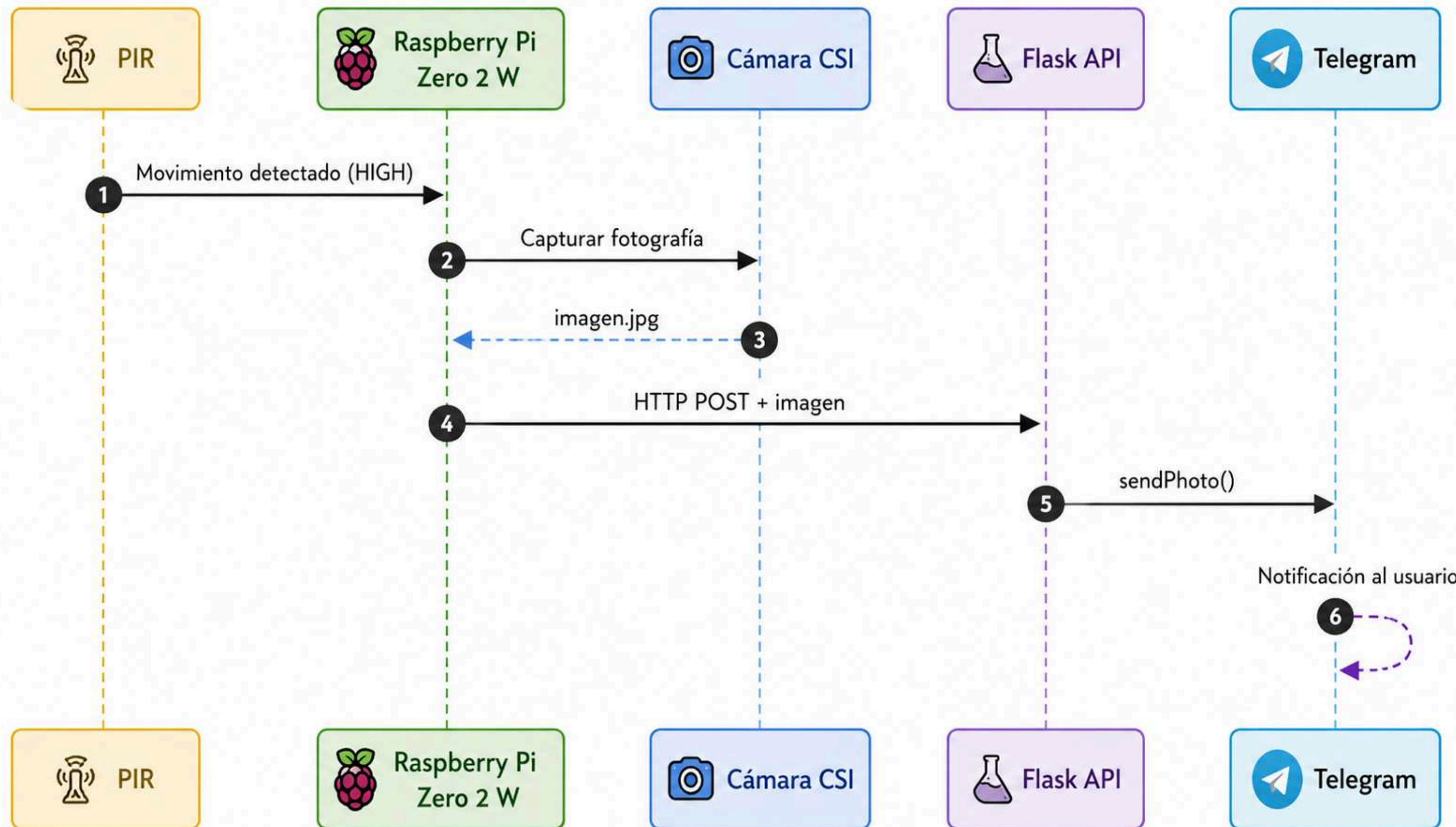
REFERENCIAS

- RFC 791 - Internet Protocol (IPv4)
- RFC 793 - Transmission Control Protocol (TCP)
- RFC 826 - Address Resolution Protocol (ARP)
- IEEE 802.11 - Wireless LAN
- IEEE 802.11-2016 - MAC Bridges (802.1D)
- Fielding, R. - Architectural Styles and the Design of Network-based Software Architectures (REST)

Topología Plana (Flat Network) en Capa 2: Todos los nodos sensores y el servidor operan en el mismo dominio de broadcast (subred IPv4 /24). Esto elimina la necesidad de hardware de enrutamiento complejo, reduciendo costos y optimizando la resolución de direcciones (ARP).

Acceso al Medio Inalámbrico: La conectividad se gestiona bajo el estándar IEEE 802.11n (WLAN en 2.4 GHz). Se emplea el protocolo CSMA/CA para evitar colisiones en el aire cuando la Raspberry Pi y el ESP32 intentan transmitir simultáneamente.

Marco Teórico - Diagrama de Flujo



Detección de Hardware: El sensor PIR detecta un cambio térmico y envía una señal en alto (HIGH) al microcontrolador/microprocesador.

Captura y Petición Interna: El nodo perceptor captura la fotografía y genera una solicitud HTTP POST (multipart/form-data) dirigida al puerto 5001 del servidor Flask en la red de área local (LAN).

Procesamiento del Middleware: El servidor recibe el evento asíncrono, valida la carga útil (imagen) y prepara la estructura de datos.

Notificación Externa (WAN): Flask consume la API de Telegram mediante un canal HTTPS, emitiendo la alerta en tiempo real al dispositivo móvil del administrador de la red.

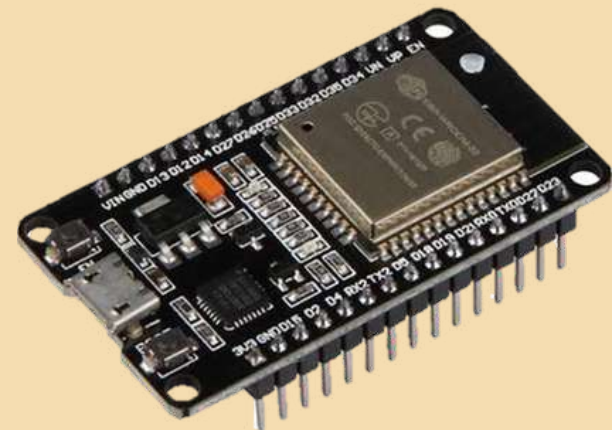
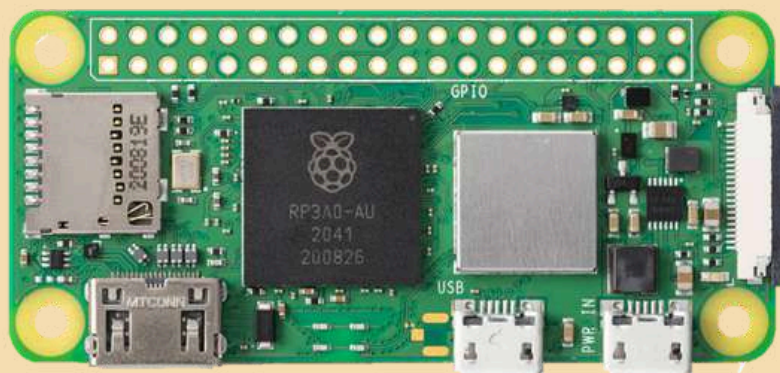
Marco Teórico

¿Por qué usamos dos nodos?

- **1. Redundancia y tolerancia a fallos:**

En sistemas de seguridad, depender de un único dispositivo es arriesgado porque genera un punto único de fallo.

- Si una cámara pierde conexión, se recalienta o falla el sensor PIR, se crea un área sin vigilancia.
- Al contar con dos equipos distintos (Raspberry Pi con procesador ARM y ESP32 con microcontrolador), aseguramos que si uno deja de funcionar, el otro continúa operando. Esto convierte al sistema en más confiable y resistente.



- **2. Escalabilidad y arquitectura distribuida**

El uso de dos cámaras demuestra que el servidor Flask no está limitado a un solo dispositivo, sino que actúa como un dispatcher, capaz de gestionar múltiples fuentes de datos.

- Esta configuración permite añadir cámaras económicas (ESP32) en diferentes puntos estratégicos.
- Todas se centralizan en el mismo servidor, lo que facilita la expansión del sistema y reduce costos de implementación.



Flask

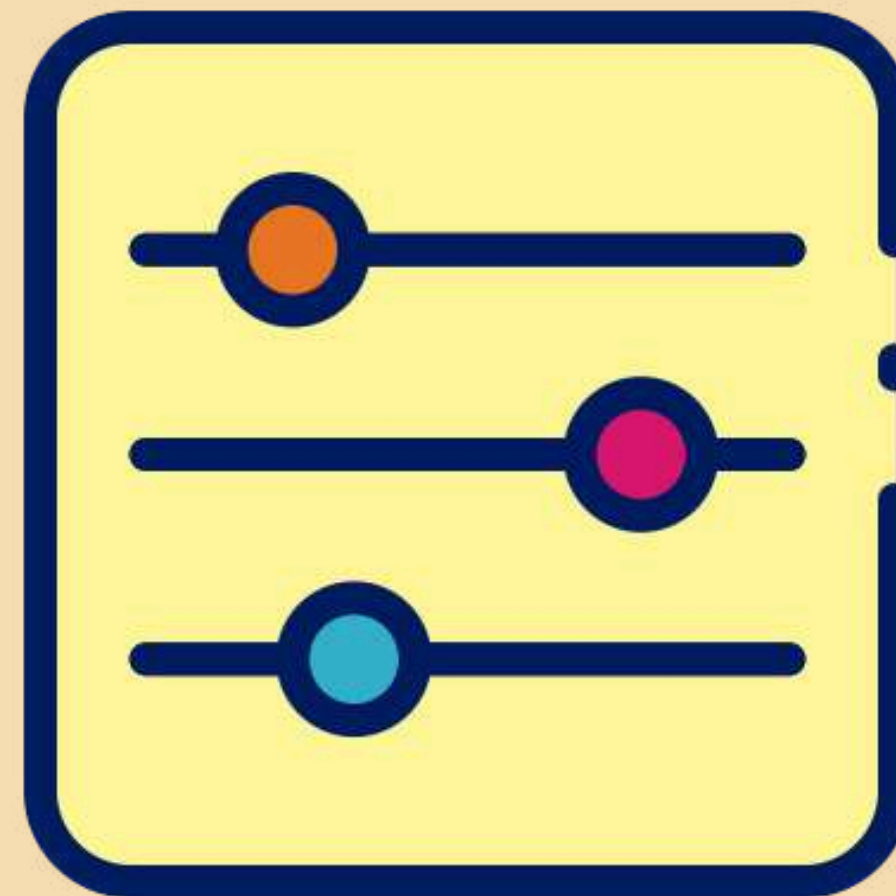
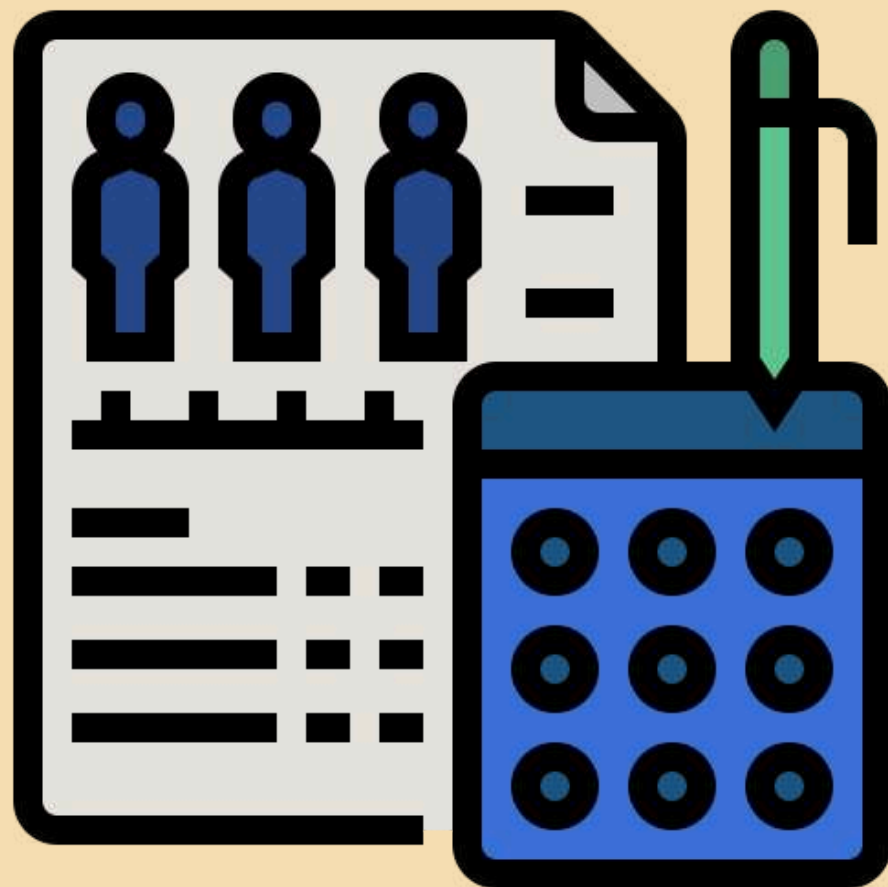
web development,
one drop at a time

Marco Teórico

¿Por qué usamos dos nodos?

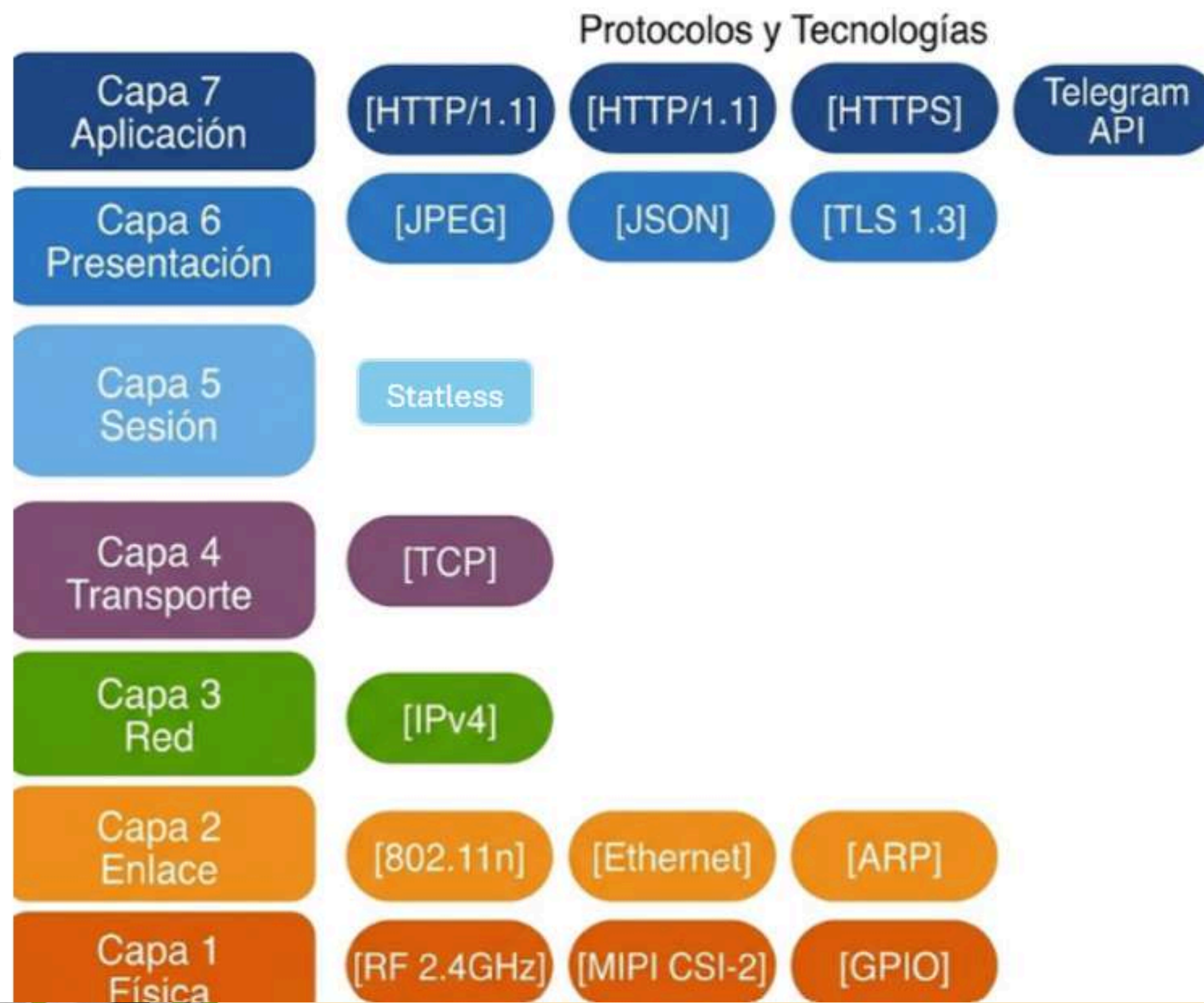
- **3. El motivo Principal**

La fase actual del proyecto es una prueba de concepto para evaluar el rendimiento, consumo de red y velocidad de disparo entre la tecnología de Raspberry Pi y la de ESP32-CAM. Esto justifica tener ambos corriendo en paralelo: lo que estaremos haciendo es determinar los parámetros y métricas para ver como se comportan con los protocolos con los que fueron diseñados.



Marco Teórico – Protocolos usados

Modelo OSI - Sistema IoT Bidireccional



Capa 2 (Enlace de Datos): Se usan IEEE 802.11n (Wi-Fi 4), Ethernet y ARP. Proceso: Controla el acceso físico al medio inalámbrico mediante CSMA/CA (para evitar colisiones) y usa ARP para traducir las direcciones IP lógicas en direcciones MAC físicas de las tarjetas de red de la PC, la Raspberry Pi y el ESP32.

Capa 7 (Aplicación): Se usan HTTP/1.1, HTTPS y Telegram Bot API. Proceso: Es la lógica del sistema. Permite a los sensores enviar la imagen a Flask (POST /image), a Flask enviar la foto a Telegram (POST /sendPhoto), y a Flask consultar nuevos comandos mediante long polling (GET /getUpdates).

Capa 6 (Presentación): Se usan JPEG, JSON y TLS 1.3. Proceso: Se encarga del formato y la seguridad. Codifica la imagen capturada en JPEG, estructura los mensajes de control en texto JSON y cifra la comunicación hacia Internet con TLS 1.3 (dejando la red local en texto plano).

Capa 5: El sistema es de naturaleza "stateless" (sin estado) en la red local y delega la persistencia de la conexión a los mecanismos de HTTP (L7) y TCP (L4).

Capa 4 (Transporte): Se usa exclusivamente TCP. Proceso: Garantiza que los paquetes de la imagen y los comandos lleguen completos y en orden mediante el establecimiento de conexiones (handshakes) entre los dispositivos

Capa 3 (Red): Se usa IPv4. Proceso: Se encarga del direccionamiento lógico (ej. 192.168.0.x) y del enrutamiento. Permite que los dispositivos se encuentren en la red local y que el router haga la traducción (NAT) para salir a Internet hacia los servidores de Telegram.

FUNCIONAMIENTO

Una vez puesto en marcha el proyecto, empezemos con el funcionamiento de la camara del ESP32-CAM, podemos tomar fotos (con flash y sin flash), encender el flash, y modo alerta (flash parpadea constantemente).

TOMAR FOTO A TRAVES DE TELEGRAM:

Se envia a traves de telegram un comando para tomar foto: /foto esp32



TOMAR FOTO CON FLASH A TRAVES DE TELEGRAM:

Se envia a traves de telegram un comando para tomar foto: /foto_flash esp32-01

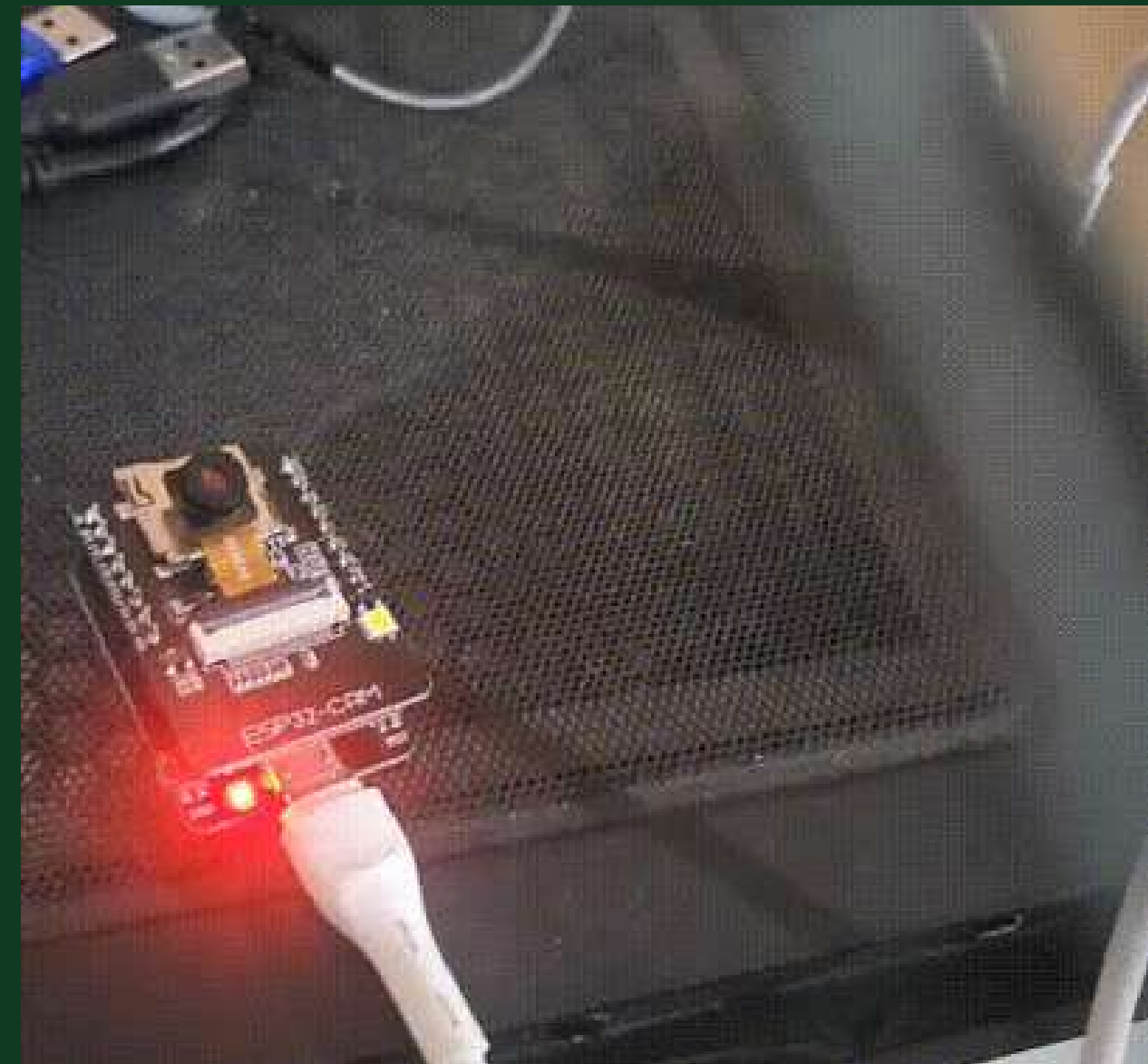


FUNCIONAMIENTO

Empezamos con el funcionamiento de la camara del ESP32-CAM, podemos tomar fotos (con flash y sin flash), encender el flash, y modo alerta (flash parpadea constantemente).

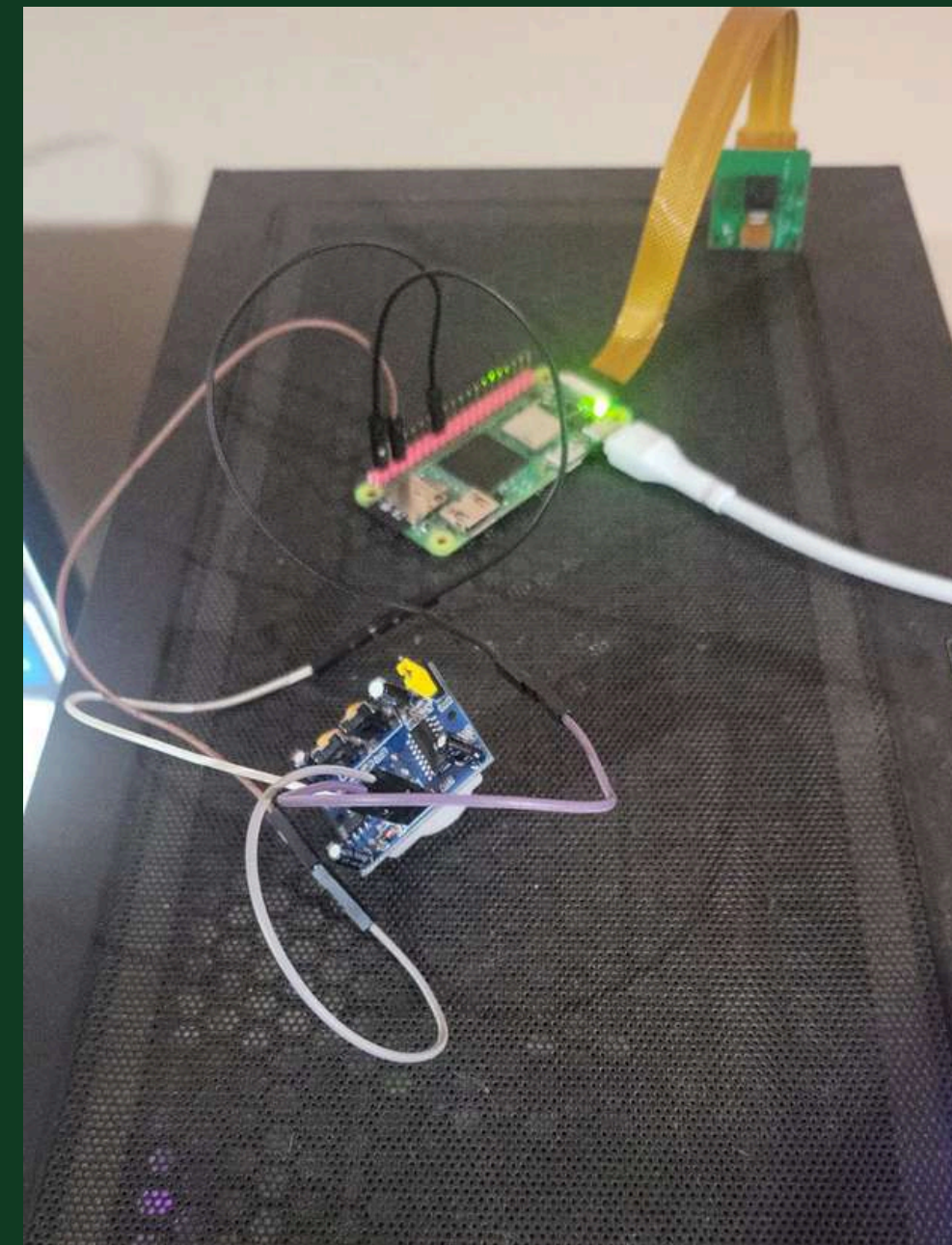
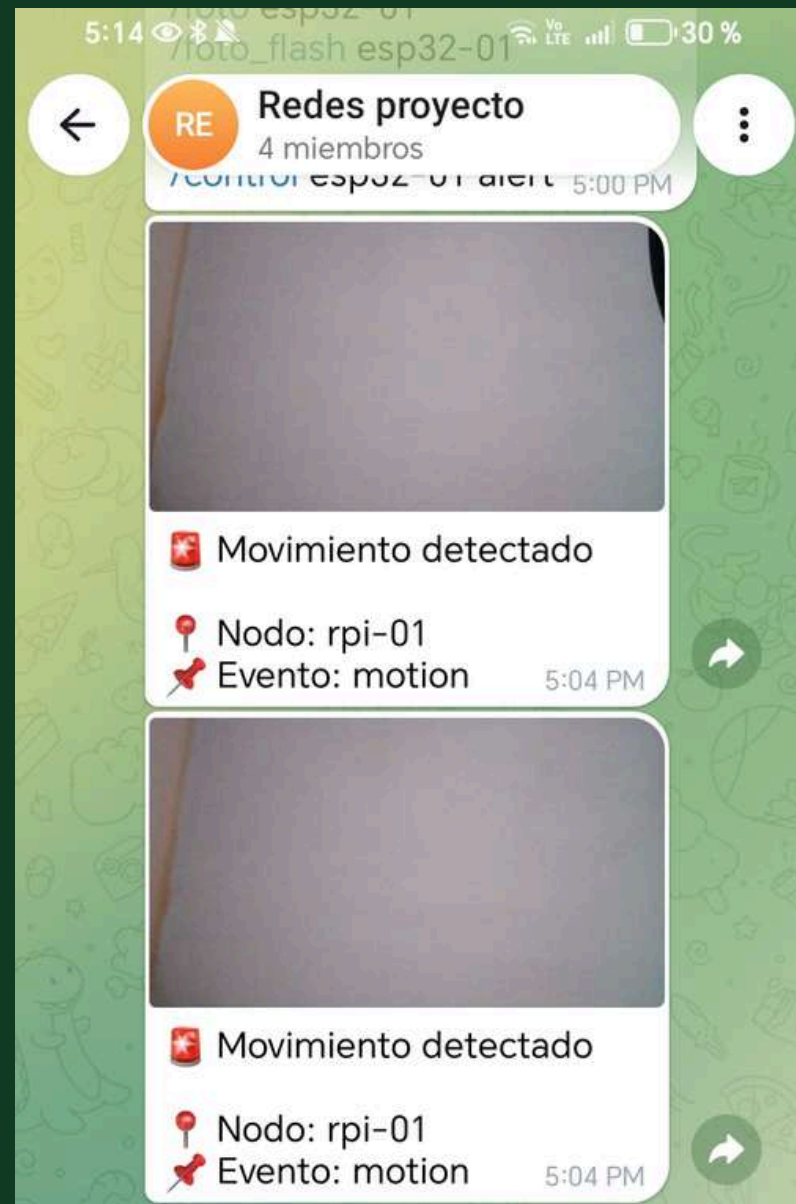
MODO ALERTA:

Se envia a traves de telegram un comando para tomar foto: `/control esp32-01 alert`



FUNCIONAMIENTO CON EL SENSOR PIR

Un sensor PIR (Infrarrojo Pasivo) detecta movimiento midiendo los cambios de calor infrarrojo en su campo de visión. Es "pasivo" porque no emite energía, solo recibe. En este caso esta conectado al nodo del raspberry zero 2w, al minimo cambio enviara la foto al telegram:



FUNCIONAMIENTO

Tenemos también un comando para evaluar las métricas en este caso es el comando /network:

Objetivo Principal:

Facilita la supervisión en tiempo real de las condiciones de la red y el estado del equipo (hardware), eliminando la necesidad de estar físicamente presentes o de acceder a través de una conexión SSH.

1. Módulo ESP32-CAM (Supervisión de la Conexión Wi-Fi)

- **Objetivo:** Revisa el estado del enlace físico entre el dispositivo y el router.
- **Métricas Principales:** Dirección IP, dirección MAC, nombre de la red (SSID) y la intensidad de la señal (RSSI).
- **Aplicación Práctica:** Resulta fundamental para identificar si la cámara está experimentando problemas de interferencia o una señal inalámbrica deficiente.

2. Raspberry Pi Zero 2W (Supervisión de Sistema y Calidad de Servicio - QoS)

- **Objetivo:** Analiza el rendimiento y calidad de la conexión de red (QoS), además de monitorear el consumo de recursos dentro del sistema operativo Linux.
- **Métricas de Red:** Registra la latencia, la pérdida de paquetes y el Jitter (las fluctuaciones en el tiempo de recepción de los datos). Esto se hace con el fin de garantizar una transmisión de imágenes constante y fluida.
- **Métricas de Hardware:** Verifica el nivel de uso de la CPU, la memoria RAM y la temperatura del dispositivo. Este control es vital para prevenir caídas de rendimiento (throttling) o apagados repentinos causados por el exceso de calor.



Latencia para el comando /status rpi-01

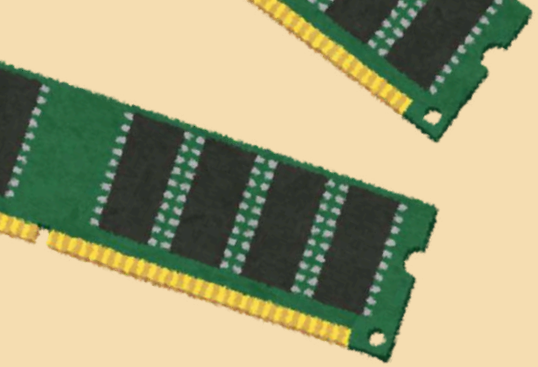
```
<
> Frame 145: Packet, 182 bytes on wire (1456 bits), 182 bytes captured (1456 bits) on interface \Device\NPF_{D2E7D52E-E4F9-47A4-BB0A-83C3465DC053}, id 0
> Ethernet II, Src: RaspberryPi_5a:fc:5e (88:a2:9e:5a:fc:5e), Dst: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf)
> Internet Protocol Version 4, Src: 192.168.0.12, Dst: 192.168.0.10
> Transmission Control Protocol, Src Port: 5001, Dst Port: 57710, Seq: 167, Ack: 207, Len: 128
> [2 Reassembled TCP Segments (294 bytes): #144(166), #145(128)]
▼ Hypertext Transfer Protocol
  > HTTP/1.1 200 OK\r\n
    Server: Werkzeug/3.1.8 Python/3.13.5\r\n
    Date: Mon, 29 Jun 2026 23:49:34 GMT\r\n
    Content-Type: application/json\r\n
  > Content-Length: 128\r\n
    Connection: close\r\n
    \r\n
    [Request in frame: 141]
    [Time since request: 17.964000 milliseconds]
    [Request URI: /status]
    [Full request URI: http://192.168.0.12:5001/status]
    File Data: 128 bytes
  > JavaScript Object Notation: application/json
```

18.5229 milisegundos

Petición Exitosa: Se observa una respuesta web correcta (HTTP/1.1 200 OK) proveniente de un servidor basado en Python (Werkzeug).

Baja Latencia: El detalle del paquete revela que el tiempo transcurrido desde que se envió la solicitud hasta que se recibió la respuesta fue de tan solo 17.96 milisegundos. Esto se alinea con el número destacado (18.5229 milisegundos), demostrando que la comunicación entre los dispositivos en la red local (IPs 192.168.0.12 y 192.168.0.10) es sumamente rápida y estable.





Throughput promedio y tamaño promedio de la imagen

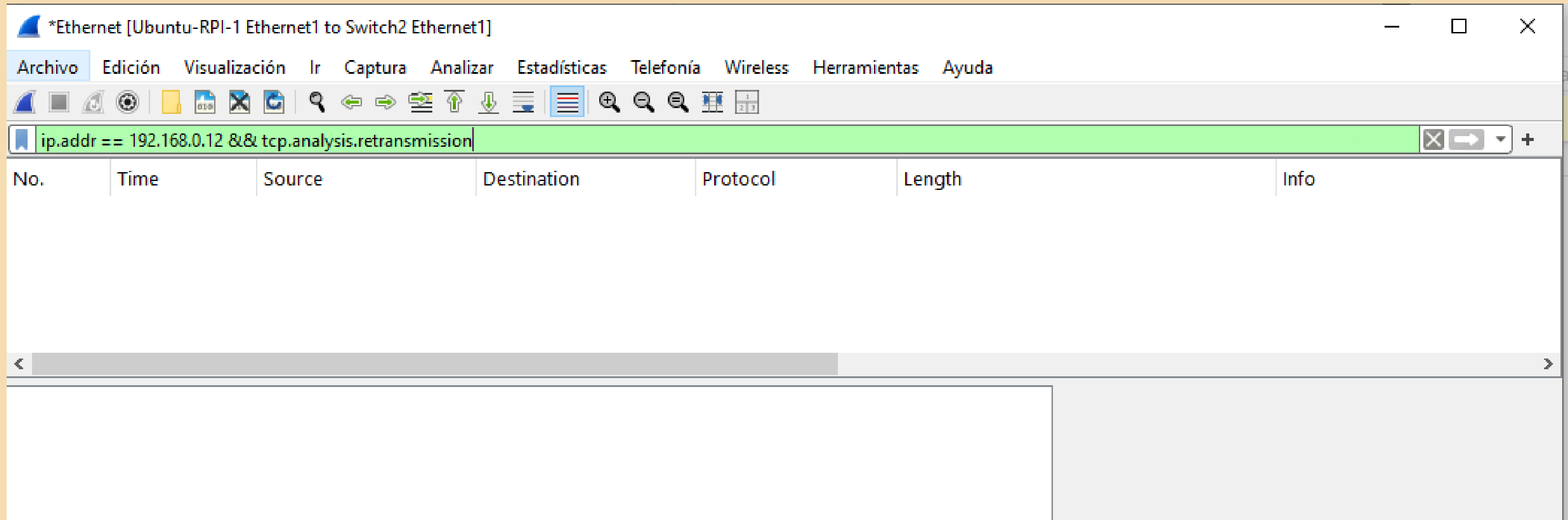
Metrica	Raspberry Pi
Latencia promedio /status	18.52 ms
Latencia promedio /foto	228.46 ms
Throughput Promedio	6.53 Mbps
Tamaño promedio de imagen	173.75 KB

El rendimiento del nodo Raspberry Pi Zero 2W demuestra una eficiencia notable plenamente alineada con los estándares de la industria de las telecomunicaciones y el Internet de las Cosas (IoT). Su latencia de monitoreo de 18.52 ms se encuentra dentro del rango óptimo (2 a 20 ms) esperado por el estándar IEEE 802.11n para comunicaciones ágiles y sin congestión en redes locales de 2.4 GHz. Por su parte, lograr todo el ciclo de captura, compresión y transmisión de una imagen en apenas 228.46 ms cumple holgadamente con los estrictos umbrales de latencia interactiva (menores a 400 ms) recomendados por las guías de Calidad de Servicio (QoS) de Cisco y la ITU-T para sistemas en tiempo real. Finalmente, el hecho de sostener un throughput de 6.53 Mbps logrando comprimir el archivo a 173.75 KB replica las mejores prácticas comerciales y operativas de consorcios como ONVIF (estándar global para cámaras de seguridad IP), asegurando que el sistema enviará alertas rápidas por Telegram sin asfixiar el ancho de banda del usuario doméstico.



Retransmisiones TCP

Esta es una de las métricas más importantes sobre la salud de la red. Al aplicar el filtro `tcp.analysis.retransmission` en Wireshark, el resultado fue nulo.



Esto significa que no hubo ninguna retransmisión TCP; todos los segmentos de datos enviados llegaron a su destino correctamente en el primer intento, sin sufrir colisiones o interferencias graves.

*Ethernet [Ubuntu-RPI-1 Ethernet1 to Switch2 Ethernet1]

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

ip.addr == 192.168.0.12

No.	Time	Source	Destination	Protocol	Length	Info
1522	8.075983	192.168.0.10	192.168.0.12	TCP	66	57797 → 5001 [SYN] S
1524	8.152107	192.168.0.12	192.168.0.10	TCP	66	5001 → 57797 [SYN, A
1525	8.152212	192.168.0.10	192.168.0.12	TCP	54	57797 → 5001 [ACK] S

< >

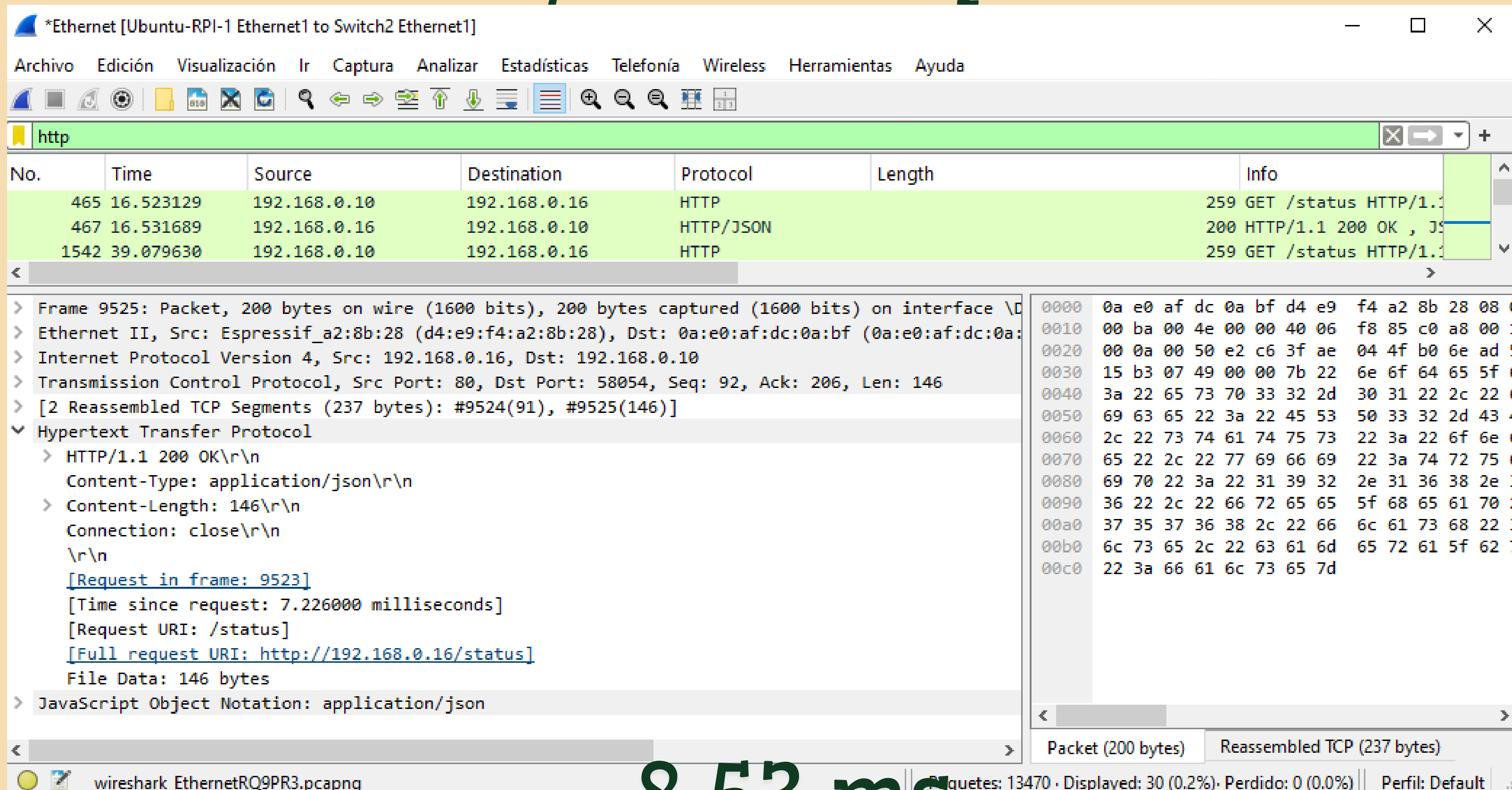
> Frame 1522: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device
> Ethernet II, Src: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf), Dst: RaspberryPi_5a:fc:5e (88:a2:9e:5a:f
> Internet Protocol Version 4, Src: 192.168.0.10, Dst: 192.168.0.12
> Transmission Control Protocol, Src Port: 57797, Dst Port: 5001, Seq: 0, Len: 0

0000 88 a2 9e 5a fc 5e 0a e0 af dc 0a bf 08 0
0010 00 34 08 78 40 00 80 06 70 e5 c0 a8 00 0
0020 00 0c e1 c5 13 89 7f 3f c8 1d 00 00 00 0
0030 fa f0 b6 0c 00 00 02 04 05 b4 01 03 03 0
0040 04 02

wireshark_Ethernet3GXOR3.pcapng Paquetes: 39817 · Displayed: 1854 (4.7%) · Perdido: 0 (0.0%) Perfil: Default

Se observa claramente el intercambio de paquetes de control [SYN], [SYN, ACK], y [ACK], lo que confirma un establecimiento de conexión TCP saludable (handshake).

Latencia para el comando /status esp32-01



The image shows a Wireshark packet capture window titled '*Ethernet [Ubuntu-RPI-1 Ethernet1 to Switch2 Ethernet1]'. The packet list on the left shows three packets: packet 465 is a GET request to /status, packet 467 is the corresponding 200 OK JSON response, and packet 1542 is another GET request. The packet details pane for packet 467 shows the HTTP response structure, including headers like Content-Type: application/json and Content-Length: 146. The packet bytes pane shows the raw data of the response, which is a JSON object. The status bar at the bottom indicates that 13470 packets were captured, 30 were displayed (0.2%), and 0 were lost (0.0%).

No.	Time	Source	Destination	Protocol	Length	Info
465	16.523129	192.168.0.10	192.168.0.16	HTTP	259	GET /status HTTP/1.1
467	16.531689	192.168.0.16	192.168.0.10	HTTP/JSON	200	HTTP/1.1 200 OK , JS
1542	39.079630	192.168.0.10	192.168.0.16	HTTP	259	GET /status HTTP/1.1

Frame 9525: Packet, 200 bytes on wire (1600 bits), 200 bytes captured (1600 bits) on interface \Device\NPF...
Ethernet II, Src: Espressif_a2:8b:28 (d4:e9:f4:a2:8b:28), Dst: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf)
Internet Protocol Version 4, Src: 192.168.0.16, Dst: 192.168.0.10
Transmission Control Protocol, Src Port: 80, Dst Port: 58054, Seq: 92, Ack: 206, Len: 146
[2 Reassembled TCP Segments (237 bytes): #9524(91), #9525(146)]
Hypertext Transfer Protocol
> HTTP/1.1 200 OK\r\n
Content-Type: application/json\r\n
Content-Length: 146\r\n
Connection: close\r\n
\r\n
[Request in frame: 9523]
[Time since request: 7.226000 milliseconds]
[Request URI: /status]
[Full request URI: http://192.168.0.16/status]
File Data: 146 bytes
JavaScript Object Notation: application/json

Packet (200 bytes) Reassembled TCP (237 bytes)

wireshark_EthernetRQ9PR3.pcapng

8.53 ms

El microcontrolador ESP32-CAM demuestra ser extremadamente rápido para procesar peticiones. Su latencia para devolver el estado (/status) es de tan solo 8.53 ms

Latencia para el comando /foto esp32-01

Logra ejecutar el ciclo completo de tomar y enviar una foto en apenas 43.16 ms.

El ESP32 es más eficiente que la Raspberry Pi porque usa FreeRTOS en lugar de un sistema operativo completo, lo que simplifica la gestión de recursos. Además, sus mensajes muy pequeños, como un JSON de 237 bytes, permiten que las transacciones TCP se realicen sin fragmentación y con mínima latencia en la conexión inalámbrica.

Wireshark packet capture showing an HTTP 200 OK response from 192.168.0.16 to 192.168.0.10. The packet details show the full request URI as `http://192.168.0.16/status` and the response body as `JavaScript Object Notation: application/json`.

No.	Time	Source	Destination	Protocol	Length	Info
465	16.523129	192.168.0.10	192.168.0.16	HTTP	259	GET /status HTTP/1.1
467	16.531689	192.168.0.16	192.168.0.10	HTTP/JSON	200	HTTP/1.1 200 OK , JS
1542	39.079630	192.168.0.10	192.168.0.16	HTTP	259	GET /status HTTP/1.1

Frame 9525: Packet, 200 bytes on wire (1600 bits), 200 bytes captured (1600 bits) on interface \Device\NPF{...}

Ethernet II, Src: Espressif_a2:8b:28 (d4:e9:f4:a2:8b:28), Dst: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf)

Internet Protocol Version 4, Src: 192.168.0.16, Dst: 192.168.0.10

Transmission Control Protocol, Src Port: 80, Dst Port: 58054, Seq: 92, Ack: 206, Len: 146

[2 Reassembled TCP Segments (237 bytes): #9524(91), #9525(146)]

Hypertext Transfer Protocol

- HTTP/1.1 200 OK\r\n
- Content-Type: application/json\r\n
- Content-Length: 146\r\n
- Connection: close\r\n
- \r\n

[Request in frame: 9523]

[Time since request: 7.226000 milliseconds]

[Request URI: /status]

[Full request URI: http://192.168.0.16/status]

File Data: 146 bytes

JavaScript Object Notation: application/json

43.16 ms



Latencia para el comando /foto esp32-01



Mètrica	ESP32-CAM
Latencia Promedio /status	8.53 ms
Latencia Promedio /foto	43.16 ms
Throughput promedio	1.58 Mbps
Tamaño promedio de la imagen	8.71 KB (8 914 bytes)

Esta velocidad se explica por el bajo peso de sus capturas. El tamaño promedio de su imagen es de solo 8.71 KB , lo que reduce drásticamente el uso de la red, requiriendo un throughput promedio de apenas 1.58 Mbps



Retransmisiones TCP

Al filtrar los datos en Wireshark, se confirma que el ESP32-CAM registró 0 retransmisiones TCP y un 0% de pérdida de paquetes. Esto certifica que el enlace de red es robusto y que las tramas se envían exitosamente al primer intento.

The image shows the Wireshark network protocol analyzer interface. The title bar reads '*Ethernet [Ubuntu-RPI-1 Ethernet1 to Switch2 Ethernet1]'. The menu bar includes Archivo, Edición, Visualización, Ir, Captura, Analizar, Estadísticas, Telefonía, Wireless, Herramientas, and Ayuda. The toolbar contains various icons for file operations, capture, analysis, and display. The filter bar at the top shows the active filter: `ip.addr == 192.168.0.16 && tcp.analysis.retransmission`. Below the filter bar is a table with the following columns: No., Time, Source, Destination, Protocol, Length, and Info. The table is currently empty. At the bottom of the interface, a status bar shows: 'This frame is a (suspected) retransmission: Label' on the left, and 'Paquetes: 57132 · Displayed: 0 (0.0%) · Perdido: 0 (0.0%) | Perfil: Default' on the right.

No.	Time	Source	Destination	Protocol	Length	Info
-----	------	--------	-------------	----------	--------	------



```
*Ethernet [Ubuntu-RPI-1 Ethernet1 to Switch2 Ethernet1]
```

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda



```
ip.addr == 192.168.0.16
```

No.	Time	Source	Destination	Protocol	Length	Info
28727	79.276449	192.168.0.10	192.168.0.16	TCP	54	58121 → 80 [ACK] Seq=
28728	79.276852	192.168.0.16	192.168.0.10	HTTP	654	HTTP/1.1 200 OK (JP
28729	79.277020	192.168.0.10	192.168.0.16	TCP	54	58121 → 80 [FIN, ACK]
28730	79.277451	192.168.0.16	192.168.0.10	TCP	60	80 → 58121 [FIN, ACK]
28731	79.277504	192.168.0.10	192.168.0.16	TCP	54	58121 → 80 [ACK] Seq=
28732	79.279007	192.168.0.16	192.168.0.10	TCP	60	80 → 58121 [ACK] Seq=
34069	100.969666	192.168.0.10	192.168.0.16	TCP	66	58129 → 80 [SYN] Seq=
34077	101.071447	192.168.0.16	192.168.0.10	TCP	62	80 → 58129 [SYN, ACK]

```
> Frame 28730: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF{...}
> Ethernet II, Src: Espressif_a2:8b:28 (d4:e9:f4:a2:8b:28), Dst: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf)
> Internet Protocol Version 4, Src: 192.168.0.16, Dst: 192.168.0.10
> Transmission Control Protocol, Src Port: 80, Dst Port: 58121, Seq: 7797, Ack: 207, Len: 0
```

0000	0a	e0	af	dc	0a	bf	d4	e9	f4	a2	8b
0010	00	28	00	73	00	00	40	06	f8	f2	c0
0020	00	0a	00	50	e3	09	17	1d	b5	60	19
0030	15	b2	88	be	00	00	ea	dd	54	ee	86

wireshark EthernetEPEXR3.pcapng

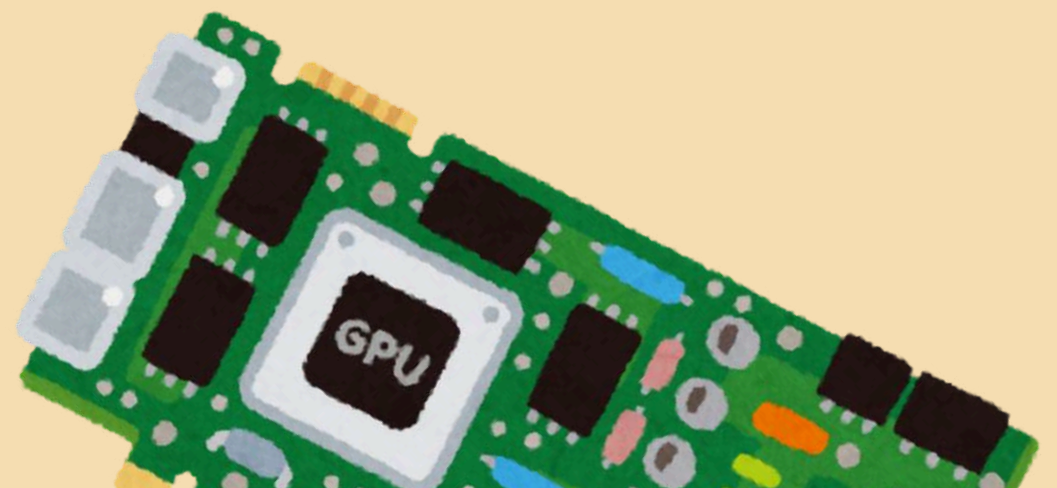
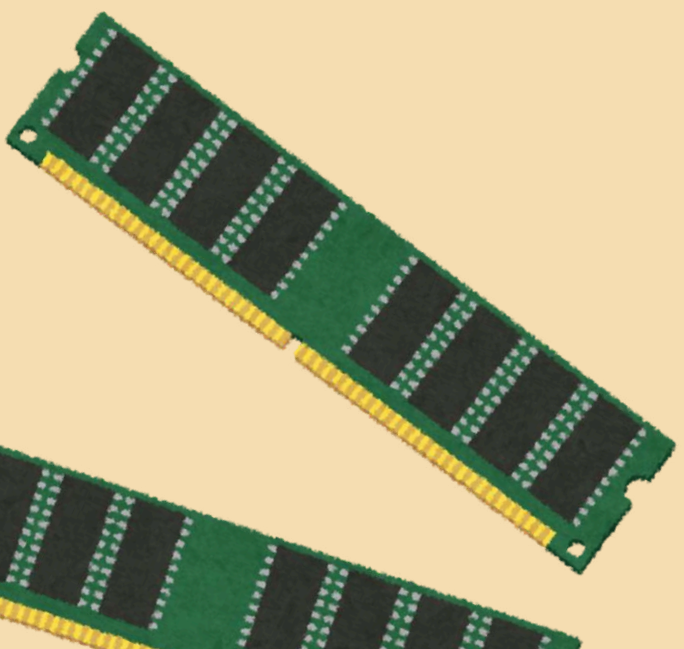
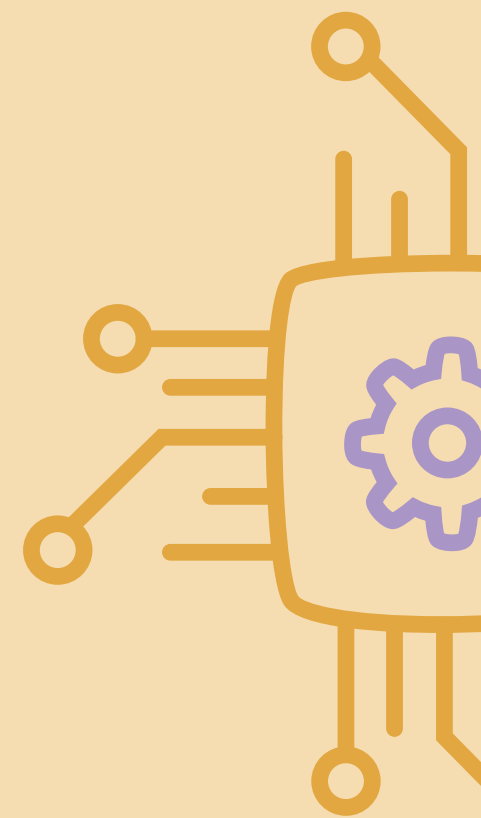
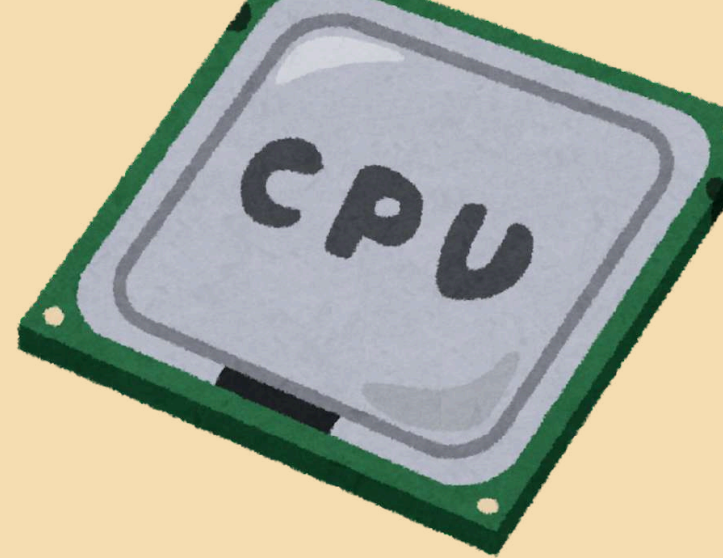
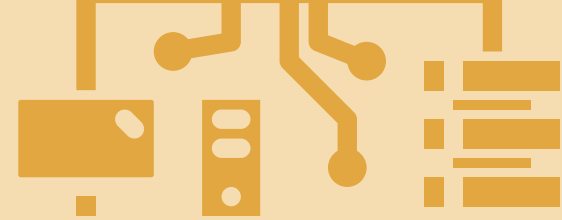
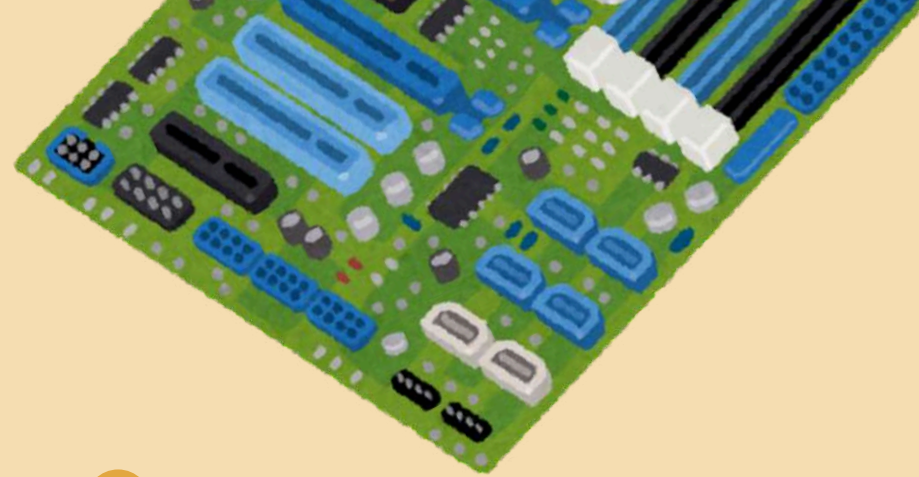
Paquetes: 57132 · Displayed: 93 (0.2%) · Perdido: 0 (0.0%) | Perfil: De

Tabla comparativa

Métrica	Raspberry Pi Zero 2W	ESP32-CAM
Latencia Promedio /status	18.52 ms	8.53 ms
Latencia Promedio /foto	228.46 ms	43.16 ms
Throughput promedio	6.53 Mbps	1.58 Mbps
Tamaño promedio de la imagen	173.75 KB (177 917.9 bytes)	8.71 KB (8 914 bytes)
Pérdida de paquetes	0%	0%
Retransmisiones TCP	0	0

La tabla final evidencia la dualidad del sistema. El ESP32-CAM resalta como la opción ultraligera y veloz, perfecta para alertas inmediatas con un mínimo consumo de red. En contraste, la Raspberry Pi procesa imágenes de mayor calidad y tamaño (173.75 KB), demandando más tiempo (228.46 ms) y más ancho de banda (6.53 Mbps), pero demostrando que ambas tecnologías logran un 100% de fiabilidad (0 pérdidas) en esta topología.

Analisis del Comando /foto rpi-01



Hypertext Transfer Protocol Capa 7 (Aplicación)

Permite la comunicación entre el servidor Flask y la Raspberry Pi mediante solicitudes HTTP, se utiliza el método GET para solicitar la captura de una imagen (/capture).

56756	502.419820	192.168.0.10	192.168.0.12	HTTP	261 GET /capture HTTP/1.1
56812	502.546239	192.168.0.12	192.168.0.10	HTTP	1330 HTTP/1.1 200 OK (JPEG JFIF image)
<					
> Frame 56756: Packet, 261 bytes on wire (2088 bits), 261 bytes captured (2088 bits) on interface \Dev					
> Ethernet II, Src: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf), Dst: RaspberryPi_5a:fc:5e (88:a2:9e:5a:fc:5					
> Internet Protocol Version 4, Src: 192.168.0.10, Dst: 192.168.0.12					
> Transmission Control Protocol, Src Port: 55399, Dst Port: 5001, Seq: 1, Ack: 1, Len: 207					
Hypertext Transfer Protocol					
GET /capture HTTP/1.1\r\n					
Request Method: GET					
Request URI: /capture					
Request Version: HTTP/1.1					
Host: 192.168.0.12:5001\r\n					
User-Agent: python-requests/2.32.5\r\n					
Accept-Encoding: gzip, deflate\r\n					
Accept: */*\r\n					
Connection: keep-alive\r\n					
Authorization: Bearer TOKEN_RPI\r\n					
X-Node-ID: rpi-01\r\n					
\r\n					
[Response in frame: 56812]					
[Full request URI: http://192.168.0.12:5001/capture]					
0000	88 a2 9e 5a fc 5e 0a e0 af dc 0a bf 08 00 45 00	...	Z.^...		E.
0010	00 f7 e7 4c 40 00 80 06 91 4d c0 a8 00 0a c0 a8	...	L@...	..M.....	
0020	00 0c d8 67 13 89 3c 77 fa fb 43 8f 80 09 50 18	...	g...<w	..C...	P.
0030	04 02 35 fc 00 00 47 45 54 20 2f 63 61 70 74 75	...	5...GE T	/captu	
0040	72 65 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73	re	HTTP/	1.1..Hos	
0050	74 3a 20 31 39 32 2e 31 36 38 2e 30 2e 31 32 3a	t:	192.1	68.0.12:	
0060	35 30 30 31 0d 0a 55 73 65 72 2d 41 67 65 6e 74	5001..	Us er-Agent		
0070	3a 20 70 79 74 68 6f 6e 2d 72 65 71 75 65 73 74	:	python -request		
0080	73 2f 32 2e 33 32 2e 35 0d 0a 41 63 63 65 70 74	s/2.32.5	..Accept		
0090	2d 45 6e 63 6f 64 69 6e 67 3a 20 67 7a 69 70 2c	-Encodin	g: gzip,		
00a0	20 64 65 66 6c 61 74 65 0d 0a 41 63 63 65 70 74	deflate	..Accept		
00b0	3a 20 2a 2f 2a 0d 0a 43 6f 6e 6e 65 63 74 69 6f	:	*/.*..C onnectio		
00c0	6e 3a 20 6b 65 65 70 2d 61 6c 69 76 65 0d 0a 41	n: keep-	alive..A		
00d0	75 74 68 6f 72 69 7a 61 74 69 6f 6e 3a 20 42 65	uthoriza	tion: Be		
00e0	61 72 65 72 20 54 4f 4b 45 4e 5f 52 50 49 0d 0a	arer TOK	EN_RPI..		
00f0	58 2d 4e 6f 64 65 2d 49 44 3a 20 72 70 69 2d 30	X-Node-I	D: rpi-0		
0100	31 0d 0a 0d 0a	1....			

La captura muestra claramente el método GET /capture HTTP/1.1. Esto confirma que el servidor Flask está actuando como cliente, solicitando correctamente la foto a la API de la Raspberry Pi alojada en el puerto 5001.

Transmission Control Protocol, Src Port: 55399, Dst Port: 5001, Seq: 1, Ack: 1, Len: 207

Source Port: 55399

Destination Port: 5001

[Stream index: 51]

[Stream Packet Number: 4]

✓ [Conversation completeness: Complete, WITH_DATA (31)]

..0. = RST: Absent

...1 = FIN: Present

.... 1... = Data: Present

.... .1.. = ACK: Present

.... ..1. = SYN-ACK: Present

.... ...1 = SYN: Present

[Completeness Flags: ·FDASS]

[TCP Segment Len: 207]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 1014495995

[Next Sequence Number: 208 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 1133477897

0101 = Header Length: 20 bytes (5)

✓ Flags: 0x018 (PSH, ACK)

000. = Reserved: Not set

...0 = Accurate ECN: Not set

.... 0... = Congestion Window Reduced: Not set

.... .0.. = ECN-Echo: Not set

.... ..0. = Urgent: Not set

.... ...1 = Acknowledgment: Set

.... 1... = Push: Set

....0.. = Reset: Not set

....0. = Syn: Not set

....0 = Fin: Not set

[TCP Flags:AP...]

Window: 1026

[Calculated window size: 262656]

[Window size scaling factor: 256]

Checksum: 0x35fc [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

✓ [Timestamps]

[Time since first frame in this TCP stream: 86.332000 milliseconds]

[Time since previous frame in this TCP stream: 104.000 microseconds]

✓ [SEQ/ACK analysis]

[iRTT: 86.228000 milliseconds]

[Bytes in flight: 207]

[Bytes sent since last PSH flag: 207]

[Client Contiguous Streams: 1]

[Server Contiguous Streams: 1]

TCP payload (207 bytes)

Transmission Control Protocol

Capa 4 (Transporte)

Permite la comunicación entre el servidor Flask y la Raspberry Pi mediante solicitudes HTTP, se utiliza el método GET para solicitar la captura de una imagen (/capture).

Wireshark nos permite ver los "engranajes" de la conexión segura. Destaca el uso del puerto destino 5001 y que las banderas (Flags) Push (PSH) y Acknowledgment (ACK) están activadas. Esto le dice a la red: "entrega estos datos inmediatamente y confírmame que llegaron".

Internet Protocol Version 4

Capa 3 (Red)

Proporciona el direccionamiento lógico y el enrutamiento de los paquetes IP entre el servidor Flask y la Raspberry Pi, permitiendo el transporte de los segmentos TCP a través de la red.

▼ Internet Protocol Version 4, Src: 192.168.0.10, Dst: 192.168.0.12

0100 = Version: 4

.... 0101 = Header Length: 20 bytes (5)

> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

Total Length: 247

Identification: 0xe74c (59212)

> 010. = Flags: 0x2, Don't fragment

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 128

Protocol: TCP (6)

Header Checksum: 0x914d [validation disabled]

[Header checksum status: Unverified]

Source Address: 192.168.0.10

Destination Address: 192.168.0.12

[Stream index: 13]

Aquí validamos el direccionamiento lógico. La captura confirma que el paquete viaja con la IP de origen 192.168.0.10 (el servidor Flask) hacia la IP de destino 192.168.0.12 (la Raspberry Pi). Además, indica que el paquete no necesita dividirse (bandera Don't fragment activa).

Ethernet II

Capa 2 (Enlace de datos)

Proporciona el direccionamiento físico mediante direcciones MAC y permite la transmisión de tramas Ethernet entre el servidor Flask y la Raspberry Pi dentro de la red local.

```
▼ Ethernet II, Src: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf), Dst: RaspberryPi_5a:fc:5e (88:a2:9e:5a:fc:5e)
  ▼ Destination: RaspberryPi_5a:fc:5e (88:a2:9e:5a:fc:5e)
    .... ..0. .... = LG bit: Globally unique address (factory default)
    .... ..0 .... = IG bit: Individual address (unicast)
  ▼ Source: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf)
    .... ..1. .... = LG bit: Locally administered address (this is NOT the factory default)
    .... ..0 .... = IG bit: Individual address (unicast)
  Type: IPv4 (0x0800)
  [Stream index: 17]
```

Pasamos del mundo lógico al mundo físico (hardware). Wireshark decodifica las direcciones MAC. Lo crucial es ver cómo la trama se entrega directamente a la tarjeta de red del destino, identificada explícitamente por el fabricante como RaspberryPi_5a:fc:5e.



Analisis del Comando /control esp32- 01 flash_on

Pasamos de analizar cómo se pide una foto, a cómo se envía una orden (encender el flash) al nodo ESP32-CAM.

Hypertext Transfer Protocol Capa 7 (Aplicación)

La comunicación utiliza el protocolo HTTP/1.1 mediante el método POST, empleado para enviar información al servidor. La solicitud está dirigida al recurso /control del ESP32 (192.168.0.16), el cual recibe comandos de control desde la PC. A diferencia del análisis anterior (que usaba GET), aquí la captura muestra un HTTP POST dirigido a la IP del ESP32 (192.168.0.16).

```
▼ [2 Reassembled TCP Segments (282 bytes): #27(259), #28(23)]
  [Frame: 27, payload: 0-258 (259 bytes)]
  [Frame: 28, payload: 259-281 (23 bytes)]
  [Segment count: 2]
  [Reassembled TCP length: 282]
  [Reassembled TCP Data [...]: 504f5354202f636f6e7472666c20485454502f312e310d0a486f73743a203139322e3136382e302e31360d0a557365722d41676556e743a20707974686f6e2d72657175657374732f322e33...]
▼ Hypertext Transfer Protocol
  ▼ POST /control HTTP/1.1\r\n
    Request Method: POST
    Request URI: /control
    Request Version: HTTP/1.1
    Host: 192.168.0.16\r\n
    User-Agent: python-requests/2.32.5\r\n
    Accept-Encoding: gzip, deflate\r\n
    Accept: */*\r\n
    Connection: keep-alive\r\n
    Authorization: Bearer TOKEN_ESP32\r\n
    X-Node-ID: esp32-01\r\n
    > Content-Length: 23\r\n
    Content-Type: application/json\r\n
    \r\n
    [Response in frame: 31]
    [Full request URI: http://192.168.0.16/control]
    File Data: 23 bytes
▼ JavaScript Object Notation: application/json
  ▼ Object
    ▼ Member: command
      [Path with value: /command:flash_on]
      [Member with value: command:flash_on]
      String value: flash_on
      Key: command
      [Path: /command]
```

Lo más importante de esta captura es que Wireshark decodifica la carga útil (Payload) mostrándonos exactamente el texto JSON que viaja por la red: {"command": "flash_on"}. Es la prueba empírica de la comunicación bidireccional.



Transmission Control Protocol Capa 4 (Transporte)

El protocolo TCP proporciona una comunicación confiable entre la PC y el ESP32. En este segmento, la PC utiliza el puerto origen 58431, asignado dinámicamente por el sistema operativo, mientras que el ESP32 escucha las solicitudes HTTP en el puerto 80.

Esta captura analiza la capa de transporte utilizando el protocolo TCP, el cual asegura una comunicación confiable entre la computadora (PC) y el dispositivo ESP32. En los detalles de Wireshark se observa que la PC origina la solicitud desde un puerto asignado dinámicamente (Src Port: 58431), mientras que el ESP32 está a la escucha en el puerto HTTP estándar (Dst Port: 80). La trama transmite una carga de datos de 23 bytes y mantiene activadas las banderas de control "Push" (PSH) y "Acknowledgment" (ACK).

```
Transmission Control Protocol, Src Port: 58431, Dst Port: 80, Seq: 260, Ack: 1, Len: 23
Source Port: 58431
Destination Port: 80
[Stream index: 4]
[Stream Packet Number: 5]
[Conversation completeness: Complete, WITH_DATA (31)]
  ..0. .... = RST: Absent
  ...1 .... = FIN: Present
  .... 1... = Data: Present
  .... .1.. = ACK: Present
  .... ..1. = SYN-ACK: Present
  .... ...1 = SYN: Present
  [Completeness Flags: ·FDASS]
[TCP Segment Len: 23]
Sequence Number: 260      (relative sequence number)
Sequence Number (raw): 153312808
[Next Sequence Number: 283      (relative sequence number)]
Acknowledgment Number: 1      (relative ack number)
Acknowledgment number (raw): 3754845964
0101 .... = Header Length: 20 bytes (5)
[Flags: 0x018 (PSH, ACK)]
  000. .... .... = Reserved: Not set
  ...0 .... .... = Accurate ECN: Not set
  .... 0... .... = Congestion Window Reduced: Not set
  .... .0.. .... = ECN-Echo: Not set
  .... ..0. .... = Urgent: Not set
  .... ...1 .... = Acknowledgment: Set
  .... .... 1... = Push: Set
  .... .... .0.. = Reset: Not set
  .... .... ..0. = Syn: Not set
  .... .... ...0 = Fin: Not set
  [TCP Flags: .....AP...]
Window: 64240
[Calculated window size: 64240]
[Window size scaling factor: -2 (no window scaling used)]
Checksum: 0x0831 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
  [Time since first frame in this TCP stream: 33.491000 milliseconds]
  [Time since previous frame in this TCP stream: 33.000 microseconds]
[SEQ/ACK analysis]
  [iRTT: 33.390000 milliseconds]
  [Bytes in flight: 282]
  [Bytes sent since last PSH flag: 23]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (23 bytes)
TCP segment data (23 bytes)
```


Capa 3 – Red (IPv4)

En este paquete, IPv4 identifica al dispositivo emisor (192.168.0.10) y al dispositivo receptor (192.168.0.16) dentro de la red local, encapsulando el segmento TCP que posteriormente transporta el mensaje HTTP.

```
✓ Ethernet II, Src: Realtek-UTP-Card (08:00:27:00:00:01), Dst: Espresst1_02:00:20:04:12:14
✓ Internet Protocol Version 4, Src: 192.168.0.10, Dst: 192.168.0.16
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ✓ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 63
    Identification: 0xbe57 (48727)
    ✓ 010. .... = Flags: 0x2, Don't fragment
        0... .... = Reserved bit: Not set
        .1... .... = Don't fragment: Set
        ..0. .... = More fragments: Not set
        ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0xbaf6 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.0.10
    Destination Address: 192.168.0.16
    [Stream index: 3]
```

Se enfoca en el protocolo IPv4, encargado del direccionamiento lógico y el enrutamiento dentro de la red local. La captura identifica de manera precisa las direcciones IP involucradas: la dirección de origen es 192.168.0.10 y la de destino es 192.168.0.16. Además, el campo de protocolo ("Protocol") tiene un valor de 6, lo que confirma que IPv4 está encapsulando un segmento TCP para transportar el mensaje HTTP subyacente.

Capa 2 - Enlace de Datos (Ethernet II)

Ethernet II se encarga de transportar la información entre los dispositivos de la red local utilizando sus direcciones MAC y de indicar, mediante el campo Type, qué protocolo está encapsulado en la carga útil de la trama.

```
> Frame 28: Packet, 77 bytes on wire (616 bits), 77 bytes captured (616 bits) on interface \Device\NPF_{D2E7D52E-E4F9-47A4-BB0A-83C3465DC053}, id 0
▼ Ethernet II, Src: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf), Dst: Espressif_a2:8b:28 (d4:e9:f4:a2:8b:28)
  ▼ Destination: Espressif_a2:8b:28 (d4:e9:f4:a2:8b:28)
    .... ..0. .... = LG bit: Globally unique address (factory default)
    .... ..0 .... = IG bit: Individual address (unicast)
  ▼ Source: 0a:e0:af:dc:0a:bf (0a:e0:af:dc:0a:bf)
    .... ..1. .... = LG bit: Locally administered address (this is NOT the factory default)
    .... ..0 .... = IG bit: Individual address (unicast)
  Type: IPv4 (0x0800)
  [Stream index: 3]
> Internet Protocol Version 4, Src: 192.168.0.10, Dst: 192.168.0.16
> Transmission Control Protocol, Src Port: 58431, Dst Port: 80, Seq: 260, Ack: 1, Len: 23
> [2 Reassembled TCP Segments (282 bytes): #27(259), #28(23)]
> Hypertext Transfer Protocol
> JavaScript Object Notation: application/json
```

Aquí se examina la capa de enlace de datos, la cual gestiona el transporte físico de la información a través de la red local utilizando direcciones MAC. La lectura de Wireshark muestra la dirección MAC de origen (0a:e0:af:dc:0a:bf) y la dirección MAC de destino asociada al fabricante Espressif (d4:e9:f4:a2:8b:28). Finalmente, el campo "Type" registra el valor 0x0800, lo que le indica a la red que la carga útil encapsulada en esta trama Ethernet es un paquete IPv4.

CONCLUSIONES

- **Viabilidad del Edge Computing:** Se demostró que el uso de dispositivos de bajo costo (Raspberry Pi y ESP32-CAM) procesando datos en el "borde" es una alternativa eficiente, accesible y viable frente a la inseguridad ciudadana.
- **Estabilidad de Red Comprobada:** El análisis de tráfico (Wireshark) evidenció una red altamente estable con 0% de pérdida de paquetes y nulas retransmisiones TCP, garantizando un uso eficiente del ancho de banda y del medio inalámbrico.
- **Aislamiento y Escalabilidad Colaborativa:** El desacoplamiento lógico mediante el servidor Flask local garantizó que la capa de percepción no quede expuesta a Internet (validado vía escaneo de puertos), logrando un ecosistema open-source seguro, modular y listo para ser expandido por la comunidad académica.

...

...



THYNK UNLIMITED

INTERNET OF THINGS

(IOT)

└

└

GRACIAS

